



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по учебной практике

УП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции
программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ Паркачев Ратмир Александрович
(подпись) (фамилия, имя, отчество)

Группы ИПо-32

Руководитель практической подготовки от профильной организации

Преподаватель _____
(должность)

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

МП «26» июня 2026 года

Руководитель практической подготовки от института

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

«27» июня 2026 года

Пермь, 2026 год

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	2
РАЗДЕЛ 1. РАЗРАБОТКА ТРЕБОВАНИЯ К ПРОГРАММНЫМ МОДУЛЯМ НА ОСНОВЕ АНАЛИЗА ПРОЕКТНОЙ И ТЕХНИЧЕСКОЙ ДОКУМЕНТАЦИИ НА ПРЕДМЕТ ВЗАИМОДЕЙСТВИЯ КОМПОНЕНТ.....	4
1.1. Сбор информации о существующем состоянии продукта.....	5
1.2. Разработка технического задания.....	6
РАЗДЕЛ 2. ИНТЕГРАЦИЯ МОДУЛЕЙ В ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ.....	8
2.1. Процесс разработки программного обеспечения.....	8
РАЗДЕЛ 3. ОТЛАДКА ПРОГРАММНОГО МОДУЛЯ ИСПОЛЬЗОВАНИЕМ СПЕЦИАЛИЗИРОВАННЫХ ПРОГРАММНЫХ СРЕДСТВ.....	13
3.1. Процесс отладки ПО.....	13
РАЗДЕЛ 4. ОСУЩЕСТВЛЯТЬ РАЗРАБОТКУ ТЕСТОВЫХ НАБОРОВ И ТЕСТОВЫХ СЦЕНАРИЕВ ДЛЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ.....	15
4.1. Процесс тестирования ПО.....	15
РАЗДЕЛ 5. ИНСПЕКТИРОВАНИЕ КОМПОНЕНТ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ НА ПРЕДМЕТ СООТВЕТСТВИЯ СТАНДАРТАМ КОДИРОВАНИЯ.....	19
5.1. Процесс инспектирования компонент программного обеспечения.....	19
ЗАКЛЮЧЕНИЕ.....	21
Приложение 1.....	26

ВВЕДЕНИЕ

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Учебная практика направлена на формирование у обучающихся первоначальных практических профессиональных умений в рамках модулей ППССЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем

В настоящей работе представлен отчет о прохождении учебной практики студента-практиканта. Основными видами деятельности являлись: Разработка программного обеспечения; выполнение отладки программного модуля с использованием специализированных программных средств; осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения; уметь производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

Разрабатывать требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент;

Выполнять интеграцию модулей в программное обеспечение;

Выполнять отладку программного модуля с использованием специализированных программных средств;

Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения.

Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;

Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности

Планировать и реализовывать собственное профессиональное и личностное развитие;

Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами

Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста.

Использовать информационные технологии в профессиональной деятельности

Пользоваться профессиональной документацией на государственном и иностранном языках

РАЗДЕЛ 1. РАЗРАБОТКА ТРЕБОВАНИЯ К ПРОГРАММНЫМ МОДУЛЯМ НА ОСНОВЕ АНАЛИЗА ПРОЕКТНОЙ И ТЕХНИЧЕСКОЙ ДОКУМЕНТАЦИИ НА ПРЕДМЕТ ВЗАИМОДЕЙСТВИЯ КОМПОНЕНТ

1.1. Сбор информации о существующем состоянии продукта

Постановка задачи

Целью проекта является разработка приложения-менеджера задач, демонстрирующего применение объектно-ориентированного программирования, архитектурного паттерна MVVM, работу с коллекциями и LINQ, привязку данных в WPF, сериализацию/десериализацию JSON, а также модульное тестирование.

Описание бизнес-процесса «AS IS» (как есть)

До разработки приложения управление задачами осуществлялось вручную — с помощью записей в блокноте, текстовых файлов или электронных таблиц. Это приводило к следующим проблемам: отсутствие централизованного хранилища задач; невозможность быстрого поиска и фильтрации; отсутствие визуального контроля сроков выполнения; сложность отслеживания статуса выполнения; отсутствие статистики по выполненным задачам.

Описание процесса «TO BE» (как должно быть)

После разработки приложения TaskManager пользователь получает следующие возможности: создание, редактирование и удаление задач через графический интерфейс; назначение приоритетов (Низкий, Средний, Высокий) и статусов (Новая, В работе, Завершена); установка сроков выполнения с автоматическим выделением просроченных задач; поиск задач по названию и описанию; фильтрация по статусу; сортировка по приоритету

и сроку; пометка важных задач; сохранение и загрузка данных в формате JSON; отображение статистики (всего, завершено, просрочено, важных, процент выполнения).

Сравнительный анализ «AS IS» и «TO BE»

В ходе сравнительного анализа установлено, что разработанное приложение устраняет все выявленные недостатки ручного управления задачами. Реализовано централизованное хранение данных в оперативной памяти с возможностью персистентного сохранения в JSON. Автоматизированы операции поиска, фильтрации и сортировки, ранее выполнявшиеся вручную. Внедрено визуальное цветовое кодирование приоритетов, статусов и просроченности, что повышает наглядность. Добавлена панель статистики в реальном времени, ранее отсутствовавшая.

1.2. Разработка технического задания

Основные требования к техническому заданию

Техническое задание разработано в соответствии с ГОСТ 19.201-78 «ЕСПД. Техническое задание. Требования к содержанию и оформлению» и ГОСТ 34.602-89 «Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы».

Структура технического задания

Структура технического задания включает: общие сведения (наименование, основание для разработки, наименование разработчика); назначение и цели создания (функциональное и эксплуатационное назначение); требования к программе (функциональные требования, требования к надёжности, условия эксплуатации); требования к программной

документации; технико-экономические показатели; порядок контроля и приёмки.

Порядок документирования требований

Требования зафиксированы в пояснительной записке проекта (файл docs/Пояснительная_записка.md в репозитории) и включают функциональные требования, описание архитектуры, стек технологий и инструкцию по запуску. Документация оформлена в формате Markdown и размещена в публичном репозитории GitHub по адресу:
https://github.com/Hokpy/Tasks_Parkachev

РАЗДЕЛ 2. ИНТЕГРАЦИЯ МОДУЛЕЙ В ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

2.1. Процесс разработки программного обеспечения

Выбор модели разработки

Для данного проекта выбрана итерационная модель разработки с элементами прототипирования. Обоснование выбора: проект учебный, требования уточнялись по мере разработки; необходимость быстрой проверки концепции интерфейса (MVVM + WPF); возможность поэтапного тестирования модулей (ядро → тесты → UI); небольшой объём кода позволяет быстро вносить изменения. В отличие от каскадной модели, итерационный подход позволил на каждой итерации добавлять новый функционал (CRUD → поиск → фильтрация → сортировка → статистика → сериализация → UI) и сразу проверять его работоспособность.

Инструменты разработки

В качестве интегрированной среды разработки использована Microsoft Visual Studio 2022 Community Edition. Платформа разработки — .NET SDK 8.0. Система контроля версий — Git. Удалённый репозиторий для хранения кода — GitHub. Все инструменты являются современными и широко применяются в индустрии разработки программного обеспечения.

Среда программирования

Среда программирования — Microsoft Visual Studio 2022 Community Edition с установленной рабочей нагрузкой «Разработка классических приложений .NET». Операционная система — Windows 10/11. Для сборки и запуска проекта используется .NET CLI (dotnet build, dotnet run, dotnet test).

Диаграмма классов

Проект реализован с использованием многоуровневой архитектуры и паттерна MVVM. На уровне Core (бизнес-логика) выделены следующие классы: TaskItem — модель задачи с полями Id, Title, Description, Priority, DueDate, Status, IsImportant, CreatedAt и вычисляемым свойством IsOverdue; TaskStatistics — модель статистики с полями Total, Completed, Overdue, InProgress, New, Important и вычисляемым свойством CompletionRate; TaskService — сервис управления задачами, реализующий CRUD-операции, поиск, фильтрацию, сортировку, сохранение/загрузку JSON и статистику; TaskPriority — перечисление приоритетов (Low, Medium, High); TaskStatus — перечисление статусов (New, InProgress, Completed).

На уровне WPF (представление) выделены классы: MainWindow — главное окно приложения (XAML); TaskDialog — диалог добавления/редактирования задачи; MainViewModel — главная ViewModel, связывающая сервис с UI; BaseViewModel — базовый класс с реализацией INotifyPropertyChanged; RelayCommand — реализация ICommand для привязки кнопок; Converters — набор value converters для визуального отображения данных. На уровне Tests выделен класс TaskServiceTests, содержащий 25+ модульных тестов для TaskService.

Разработка программного модуля

Структура репозитория проекта представлена в приложении 1 и имеет следующий вид:

```
TaskManager.sln
├── TaskManager.Core/      # Библиотека классов (бизнес-логика)
│   ├── Enums/
│   │   ├── TaskPriority.cs
│   │   └── TaskStatus.cs
│   ├── Models/
│   │   ├── TaskItem.cs
│   │   └── TaskStatistics.cs
│   └── Services/
│       └── TaskService.cs
├── TaskManager.WPF/      # WPF-приложение
│   ├── ViewModels/
│   │   ├── BaseViewModel.cs
│   │   ├── RelayCommand.cs
│   │   └── MainViewModel.cs
│   └── Views/
│       ├── MainWindow.xaml
│       ├── MainWindow.xaml.cs
│       ├── TaskDialog.xaml
│       ├── TaskDialog.xaml.cs
│       └── Converters.cs
└── TaskManager.Tests/    # Модульные тесты (xUnit)
    └── TaskServiceTests.cs
```

Такое разделение называется многоуровневой архитектурой (n-tier). Бизнес-логика (Core) полностью независима от UI — её можно протестировать без запуска приложения. Проект WPF ссылается на Core, проект Tests также ссылается на Core.

Этапы разработки ПО

1. Выбор шаблона: выбран архитектурный паттерн MVVM (Model-View-ViewModel) для разделения бизнес-логики и пользовательского интерфейса. Преимущества MVVM: View не содержит бизнес-логики — только разметку; ViewModel не знает о конкретном View — легко тестировать; данные передаются через механизм привязки (Binding), а не вручную.

2. Составление схемы информационной базы: определены сущности TaskItem и TaskStatistics, перечисления TaskPriority и TaskStatus. Хранение данных осуществляется в оперативной памяти (List<TaskItem>) с возможностью сериализации в JSON через System.Text.Json.

3. Разработка форм для ввода, просмотра и редактирования данных: главное окно (MainWindow.xaml) содержит DataGrid для отображения задач, панель поиска, фильтрацию, сортировку, кнопки CRUD-операций, панель статистики. Диалог задачи (TaskDialog.xaml) содержит поля для ввода названия, описания, приоритета, статуса, срока, признака важности.

4. Документирование системы: создана пояснительная записка в формате Markdown (docs/Пояснительная_записка.md), включающая описание архитектуры, диаграмму классов, инструкцию по запуску.

5. Тестирование: разработан набор из 25+ модульных тестов на фреймворке xUnit, покрывающих все публичные методы сервиса TaskService.

6. Сопровождение системы: код размещён в публичном репозитории GitHub, что обеспечивает версионирование и возможность коллективной работы.

Этапы разработки интерфейса

1. Проектирование: определены 4 зоны главного окна — заголовок, панель управления, таблица задач, панель статистики. Разработан макет с цветовой схемой (тёмно-синий заголовок, тёмная панель статистики, цветовое кодирование строк).

2. Прототипирование: созданы XAML-разметки главного окна и диалога задачи с привязкой данных (Data Binding) к ViewModel. Использованы конвертеры для визуального отображения перечислений. Главное окно состоит из поля поиска с автоматической фильтрацией при вводе (UpdateSourceTrigger=PropertyChanged), выпадающего списка фильтра по статусу, кнопок сортировки, кнопок CRUD и DataGrid с семью колонками.

3. Стилизация: применены цветовые бейджи для приоритетов (зелёный — Низкий, жёлтый — Средний, красный — Высокий) и статусов (серый — Новая, синий — В работе, зелёный — Завершена). Реализовано выделение важных задач жёлтым фоном, просроченных — розовым фоном. Для конвертации данных использованы шесть value converters: PriorityToTextConverter, PriorityToBrushConverter, StatusToTextConverter, StatusToBrushConverter, BoolToStarConverter, BoolToVisibilityConverter.

РАЗДЕЛ 3. ОТЛАДКА ПРОГРАММНОГО МОДУЛЯ С ИСПОЛЬЗОВАНИЕМ СПЕЦИАЛИЗИРОВАННЫХ ПРОГРАММНЫХ СРЕДСТВ

3.1. Процесс отладки ПО

Отладка программного модуля проводилась с использованием встроенных средств Visual Studio 2022. Основные методы и инструменты отладки:

1. Точки останова (Breakpoints): установлены в ключевых методах TaskService (AddTask, UpdateTask, DeleteTask) и MainViewModel (RefreshTaskList) для проверки корректности передачи данных между слоями архитектуры MVVM. Точки останова позволили отследить значения переменных на каждом этапе выполнения операций.

2. Пошаговое выполнение (Step Over / Step Into): использовано для трассировки логики метода RefreshTaskList в MainViewModel, где применяется комплексная фильтрация через LINQ. Пошаговое выполнение позволило выявить и исправить ошибку в цепочке LINQ-операторов при одновременном применении фильтра по статусу и текстового поиска.

3. Окна Locals и Watch: использованы для контроля значений переменных ObservableCollection<TaskItem>, полей статистики, свойства IsOverdue при различных входных данных. В окне Watch отслеживалось вычисляемое свойство IsOverdue для задач с прошедшим сроком, что позволило убедиться в корректности логики: задача считается просроченной, если DueDate.Date < DateTime.Today и Status != Completed.

4. Обработка исключений: в классе TaskService реализована валидация входных данных (guard clauses) с генерацией соответствующих исключений:

ArgumentNullException при передаче null в методы AddTask и UpdateTask; ArgumentException при пустом названии задачи или пустом пути к файлу; KeyNotFoundException при попытке удаления или обновления несуществующей задачи; FileNotFoundException при загрузке из несуществующего файла; InvalidDataException при некорректном JSON.

5. Логирование через консоль: использован переопределённый метод ToString() в классе TaskItem для вывода отладочной информации в формате: [Priority] Title — Status (до dd.MM.yyyy). Это позволило быстро идентифицировать задачи при отладке без запуска полноценного UI.

6. Демонстрационные данные: метод LoadDemoData() в MainViewModel создаёт 5 тестовых задач с различными статусами, приоритетами и сроками (включая просроченную и завершённую задачи), что позволяет сразу после запуска проверить работу фильтрации, сортировки, визуального выделения и статистики

РАЗДЕЛ 4. ОСУЩЕСТВЛЯТЬ РАЗРАБОТКУ ТЕСТОВЫХ НАБОРОВ И ТЕСТОВЫХ СЦЕНАРИЕВ ДЛЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

4.1. Процесс тестирования ПО

Тестирование проводилось по методологии модульного (unit) тестирования с использованием фреймворка xUnit — современного инструмента для платформы .NET, являющегося стандартом де-факто для тестирования .NET-приложений.

Структура тестового проекта TaskManager.Tests

Тестовый класс TaskServiceTests содержит 25 тестовых методов, покрывающих все публичные методы сервиса TaskService. Для создания изолированных тестовых данных используются фабричные методы: CreateService() возвращает новый пустой экземпляр TaskService; MakeTask() создаёт тестовую задачу с заданными параметрами (по умолчанию: название «Тест», приоритет Medium, статус New, не важная, срок через 3 дня). Каждый тест следует принципу AAA: Arrange — подготовка данных, Act — выполнение действия, Assert — проверка результата.

Категории тестов

Добавление задач (5 тестов): проверка увеличения счётчика при добавлении; сохранение нескольких задач; валидация пустого названия (ArgumentException); валидация null (ArgumentNullException); автоматическое назначение уникального Id при Guid.Empty.

Удаление задач (3 теста): корректное удаление по Id с уменьшением списка; обработка несуществующего Id (KeyNotFoundException); очистка списка при удалении единственной задачи.

Обновление задач (2 теста): изменение полей существующей задачи; обработка обновления несуществующей задачи (KeyNotFoundException).

Поиск (5 тестов): поиск по части названия; поиск по части описания; поиск без учёта регистра; пустой запрос возвращает все задачи; отсутствие совпадений возвращает пустой список.

Фильтрация (2 теста): фильтр по статусу New возвращает только новые задачи; фильтр по статусу Completed возвращает только завершённые.

Сортировка (2 теста): по приоритету (High → Medium → Low); по сроку (ближайшие первые).

Статистика (5 тестов): корректное общее количество; подсчёт завершённых; подсчёт просроченных; процент выполнения 100% при всех завершённых; процент выполнения 0% для пустого списка; подсчёт важных задач.

Сохранение и загрузка JSON (6 тестов): создание файла на диске; восстановление ранее сохранённых задач с сохранением всех полей; сохранение/загрузка нескольких задач; обработка несуществующего файла (FileNotFoundException); валидация пустого пути (ArgumentException); замена существующих задач при загрузке.

Просроченность (3 теста): IsOverdue = true при прошедшем сроке и незавершённом статусе; IsOverdue = false при завершённом статусе (даже если срок прошёл); IsOverdue = false при будущем сроке.

Примеры unit-тестов

Тест добавления задачи с проверкой счётчика:

```
[Fact(DisplayName = "Добавление задачи увеличивает счётчик на 1")]  
public void AddTask_IncreasesCount()
```



```

{
    var service = CreateService();
    service.AddTask(MakeTask("Задача 1"));
    Assert.Single(service.GetAllTasks());
}

```

Тест валидации входных данных:

[Fact(DisplayName = "Добавление задачи с пустым названием бросает исключение")]

```

public void AddTask_EmptyTitle_ThrowsArgumentException()
{
    var service = CreateService();
    var task = MakeTask("");
    Assert.Throws<ArgumentException>(() => service.AddTask(task));
}

```

Тест сериализации и десериализации JSON:

[Fact(DisplayName = "LoadFromFile восстанавливает ранее сохранённые задачи")]

```

public void LoadFromFile_RestoresTasks()
{
    var original = CreateService();
    original.AddTask(MakeTask("Задача для сохранения",
        priority: TaskPriority.High,
        status: Core.Enums.WorkStatus.InProgress,
        isImportant: true));
    string path = Path.GetTempFileName();
    try
    {

```

```

        original.SaveToFile(path);
        var loaded = CreateService();
        loaded.LoadFromFile(path);
        var tasks = loaded.GetAllTasks();
        Assert.Single(tasks);
        Assert.Equal("Задача для сохранения", tasks[0].Title);
        Assert.Equal(TaskPriority.High, tasks[0].Priority);
    }
    finally { File.Delete(path); }
}

```

Запуск тестов осуществляется командой `dotnet test TaskManager.Tests/TaskManager.Tests.csproj` или через Visual Studio (Test → Run All Tests, Ctrl+R, A). Все 25 тестов успешно пройдены, что подтверждает корректность работы бизнес-логики и соответствие требованиям технического задания.

РАЗДЕЛ 5. ИНСПЕКТИРОВАНИЕ КОМПОНЕНТ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ НА ПРЕДМЕТ СООТВЕТСТВИЯ СТАНДАРТАМ КОДИРОВАНИЯ

5.1. Процесс инспектирования компонент программного обеспечения

Виды инспекции

В проекте проведены следующие виды инспекции: статический анализ кода (Code Review) — ручная проверка исходного кода на соответствие стандартам оформления C#; автоматическая проверка компилятором — все проекты собираются без предупреждений (Warning Level 4).

Актуальные стандарты кодирования

1. Microsoft C# Coding Conventions — официальные рекомендации Microsoft по написанию кода на C#. В проекте соблюдены правила именования: PascalCase для классов, методов, свойств; camelCase для параметров и локальных переменных; _camelCase для приватных полей. Применено использование var при очевидном типе. Использованы expression-bodied members для коротких свойств и методов (например, IsOverdue, CompletionRate, ToString).

2. SOLID-принципы объектно-ориентированного проектирования: Single Responsibility Principle — каждый класс отвечает за одну задачу (TaskService — бизнес-логика, MainViewModel — связь с UI, TaskItem — модель данных, Converters — визуальное отображение); Open/Closed Principle — расширение функциональности через добавление новых конвертеров без изменения существующего кода; Liskov Substitution Principle — RelayCommand корректно реализует интерфейс ICommand; Interface Segregation — использование минимальных интерфейсов (INotifyPropertyChanged, ICommand); Dependency Inversion — ViewModel зависит от абстракции TaskService, а не от конкретной реализации UI.

3. Паттерны проектирования: MVVM (Model-View-ViewModel) — разделение ответственности между слоями; Repository (частично) — TaskService инкапсулирует доступ к данным; Command — RelayCommand для декларативной привязки действий к UI; Factory — фабричные методы CreateService() и MakeTask() в тестах.

Программы для инспекции

Visual Studio 2022 — встроенный анализатор кода Roslyn, подсветка синтаксиса, IntelliSense, рефакторинг. Git — контроль версий, просмотр diff между коммитами. GitHub — веб-интерфейс для просмотра кода, истории изменений.

Результаты инспектирования

Код полностью документирован XML-комментариями (///) для всех публичных классов, методов и свойств. Отсутствуют «магические числа» — используются именованные константы и перечисления (TaskPriority, TaskStatus). Все публичные методы содержат проверку входных данных (guard clauses) с информативными сообщениями об ошибках. Исключения содержат контекстную информацию (например, «Задача с Id={id} не найдена»). Соблюдено единообразие отступов и форматирования (4 пробела). Названия классов, методов и свойств являются самодокументируемыми (например, GetOverdueTasks, SortByPriority, CompletionRate). Код соответствует стандартам оформления C#-программ и может быть передан на сопровождение без дополнительной доработки.

ЗАКЛЮЧЕНИЕ

В результате прохождения учебной практики с 8 по 28 июня 2026 г. были закреплены теоретические знания и расширены профессиональные умения в области разработки программного обеспечения.

По итогам практики получены следующие результаты:

1. Разработано полнофункциональное настольное приложение TaskManager на платформе .NET 8 с использованием технологии WPF и архитектурного паттерна MVVM. Приложение размещено в публичном репозитории GitHub: https://github.com/Hokpy/Tasks_Parkachev.

2. Реализована многоуровневая архитектура с разделением на модули: TaskManager.Core (бизнес-логика), TaskManager.WPF (пользовательский интерфейс), TaskManager.Tests (модульное тестирование). Модуль Core полностью независим от UI и может быть протестирован изолированно.

3. Применены современные технологии разработки: язык C# 12, платформа .NET 8, технология WPF, паттерн MVVM, LINQ для обработки коллекций, System.Text.Json для сериализации, xUnit для модульного тестирования, Git для контроля версий.

4. Разработан набор из 25 модульных тестов, обеспечивающих покрытие ключевых сценариев использования: CRUD-операции, поиск, фильтрация, сортировка, статистика, сериализация/десериализация JSON, логика просроченности. Все тесты успешно пройдены.

5. Проведено инспектирование кода на соответствие стандартам Microsoft C# Coding Conventions и принципам SOLID. Выявлено полное соответствие кода установленным требованиям: документирование XML-комментариями, валидация входных данных, самодокументируемые имена, отсутствие «магических чисел».

По окончании практики был получен практический опыт разработки требований к программным модулям на основе анализа проектной и

технической документации на предмет взаимодействия компонент; интеграции модулей в программное обеспечение; отладки программного модуля с использованием специализированных программных средств; разработки тестовых наборов и тестовых сценариев для программного обеспечения; инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Троелсен, Э. Язык программирования C# 10 и платформа .NET 6 / Э. Троелсен, Ф. Джепикс. — 10-е изд. — Санкт-Петербург: Диалектика, 2022. — 1328 с. — ISBN 978-5-907399-17-6.
2. Рихтер, Д. CLR via C#. Программирование на платформе Microsoft .NET Framework 4.5 на языке C# / Д. Рихтер. — 4-е изд. — Санкт-Петербург: Питер, 2020. — 896 с. — ISBN 978-5-496-00433-6.
3. Албахари, Д. C# 10. Справочник. Полное описание языка / Д. Албахари. — 10-е изд. — Санкт-Петербург: Символ-Плюс, 2022. — 1056 с. — ISBN 978-5-93286-233-0.
4. Макконелл, С. Совершенный код. Мастер-класс / С. Макконелл. — 2-е изд. — Москва: Русская редакция; Символ-Плюс, 2021. — 896 с. — ISBN 978-5-93286-201-9.
5. Фаулер, М. Шаблоны корпоративных приложений / М. Фаулер. — Москва: Вильямс, 2020. — 544 с. — ISBN 978-5-907203-28-2.
6. Мартин, Р. Чистый код. Создание, анализ и рефакторинг / Р. Мартин. — 2-е изд. — Санкт-Петербург: Символ-Плюс, 2022. — 464 с. — ISBN 978-5-93286-218-6.
7. Нейгел, К. C# 10 и .NET 6. Разработка для Windows, macOS, iOS, Android, watchOS, tvOS и WebAssembly / К. Нейгел. — 2-е изд. — Санкт-Петербург: Диалектика, 2022. — 1376 с. — ISBN 978-5-907399-18-3.
8. Скит, Д. C# для профессионалов. Тонкости программирования / Д. Скит. — 4-е изд. — Москва: ДМК Пресс, 2021. — 608 с. — ISBN 978-5-97060-869-4.
9. Головач, В. C#. Практический курс для решения прикладных задач / В. Головач. — Москва: Лучшие книги, 2020. — 480 с. — ISBN 978-5-93650-176-5.

- 10.Зиборов, В. Visual C# на примерах / В. Зиборов. — Москва: БХВ-Петербург, 2021. — 480 с. — ISBN 978-5-9775-4056-1.
- 11.Microsoft .NET: документация [Электронный ресурс]. — Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet>. — (Дата обращения: 23.06.2026).
- 12.C#: документация [Электронный ресурс]. — Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet/csharp>. — (Дата обращения: 23.06.2026).
- 13.ASP.NET Core: документация [Электронный ресурс]. — Режим доступа: <https://learn.microsoft.com/ru-ru/aspnet/core>. — (Дата обращения: 23.06.2026).
- 14.Entity Framework Core: документация [Электронный ресурс]. — Режим доступа: <https://learn.microsoft.com/ru-ru/ef/core>. — (Дата обращения: 23.06.2026).
- 15.VisualStudio: документация [Электронный ресурс]. — Режим доступа: <https://learn.microsoft.com/ru-ru/visualstudio>. — (Дата обращения: 23.06.2026).
- 16.NuGet: документация [Электронный ресурс]. — Режим доступа: <https://learn.microsoft.com/ru-ru/nuget>. — (Дата обращения: 23.06.2026).
- 17.WPF: документация [Электронный ресурс]. — Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf>. — (Дата обращения: 23.06.2026).
- 18.LINQ: документация [Электронный ресурс]. — Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet/csharp/linq>. — (Дата обращения: 23.06.2026).
- 19.Git: документация [Электронный ресурс]. — Режим доступа: <https://git-scm.com/doc>. — (Дата обращения: 23.06.2026).

20.Stack Overflow: вопросы и ответы по C# [Электронный ресурс]. —
Режим доступа: <https://stackoverflow.com/questions/tagged/c%23>. —
(Дата обращения: 23.06.2026).

Требование к написанию программного модуля

Задание.

Цель:

Практика работы с классами, коллекциями, файловым вводом-выводом, LINQ и основами ООП в C#.

1. Основные требования: Разработать приложение для управления задачами с возможностью:

1.1 Добавления новой задачи (название, описание, приоритет, срок выполнения, статус).

1.2 Просмотра списка всех задач.

1.3 Фильтрации задач по статусу (например, "Новая", "В процессе", "Завершена").

1.4 Поиска задач по названию или описанию.

1.5 Редактирования существующих задач.

1.6 Удаления задач.

1.7 Сохранения и загрузки задач в/из файла (JSON или XML).

1.8 Основная логика приложения должна находиться в динамической библиотеке, а пользовательский интерфейс должен представлять из себя WPF-приложение.

1.9 Для классов в библиотеке необходимо написать модульные тесты.

2. Дополнительные задания (по желанию)

2.1 Реализовать сортировку задач по приоритету или сроку выполнения.

2.2 Добавить возможность пометки задачи как "Важная" (выводить звёздочкой или цветом в консоли).

2.3 Реализовать простую статистику (например, сколько задач завершено, сколько просрочено).

Сопроводительная записка должна содержать:

- титульный лист,
- оглавление,
- диаграмму классов,
- подробное описание действий по созданию ПО (с иллюстрациями).

Исходные коды приложения и сопроводительную записку выложить на GitHub и ссылку на репозиторий прислать мне на почту